

COMP2048: Theory of Computing

Dr. Josh Arnold
Week 4

19/3/2026

Recap: Regular Languages

In the last few lectures we explored **regular languages**:

- ▶ Described by **DFAs**, **NFAs**, and **regular expressions**
- ▶ All three are equivalent: they describe the **same** class of languages
- ▶ Closed under complement, union, intersection, concatenation, star
- ▶ **Pumping Lemma**: a tool for proving languages are *not* regular

Two kinds of formalism:

- ▶ **Recognisers** (machines): DFA / NFA read a string, accept or reject
- ▶ **Descriptors**: Regular expressions describe a pattern

Recap: The DFA

A DFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$.

What are the five parts?

- ▶ Q : finite set of states
- ▶ Σ : input alphabet
- ▶ $\delta : Q \times \Sigma \rightarrow Q$: transition function
- ▶ $q_0 \in Q$: start state
- ▶ $F \subseteq Q$: accepting states

Key properties:

- ▶ **Deterministic**: exactly one transition per state per symbol
- ▶ DFA = NFA in power (but NFAs can be exponentially smaller)
- ▶ NFA acceptance: $\exists$ **at least one** accepting path

The Limits of Regular Languages

Finite automata are a very **weak** model of computation:

- ▶ A DFA has a **fixed, finite** number of states
- ▶ It has **no memory** other than which state it is in
- ▶ It cannot count, and cannot remember an unbounded amount of information

Example: $D = \{0^n 1^n \mid n \geq 0\}$ is **not regular**.

A DFA would need to count the 0s and remember that count when checking the 1s, but it only has finitely many states.

What's Next?

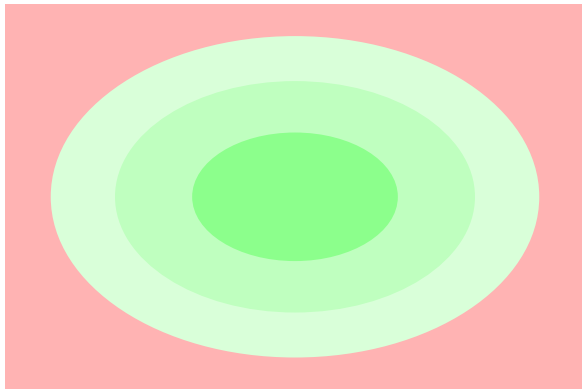
Question: What if we gave a finite automaton some form of **memory**?

What kind of languages could it recognise?

Today we will explore two new (equivalent) models:

- ▶ **Context-Free Grammars** (CFGs): a way to *generate* languages
- ▶ **Pushdown Automata** (PDAs): a way to *recognise* languages

Our Big Picture Revisited



Every regular language is context-free, but not vice versa.

Today's Plan

1. Context-Free Grammars (CFGs): definition & examples
2. Parse trees & ambiguity
3. Pushdown Automata (PDAs): definition & examples
4. The equivalence: $\text{CFG} = \text{PDA} = \text{Context-Free Languages}$
5. Converting CFGs to PDAs

Why Context-Free Grammars?

So far we have described languages using **machines** (DFAs, NFAs).

Now we take a different approach: instead of *recognising* strings, we *generate* them.

Context-free grammars appear everywhere:

- ▶ **Programming languages:** syntax of Python, Java, C
- ▶ **Compilers:** parsing source code into syntax trees
- ▶ **Natural language:** sentence structure (noun phrases, verb phrases)

Example: Grammar G_1

A **context-free grammar** is a set of substitution rules.

Here is a grammar G_1 :

$$S \rightarrow 0S1 \mid R$$

$$R \rightarrow \varepsilon$$

The “|” means “or”: $S \rightarrow 0S1 \mid R$ is shorthand for two rules:

$$S \rightarrow 0S1 \quad S \rightarrow R$$

So G_1 has **three rules** in total: $S \rightarrow 0S1$, $S \rightarrow R$, $R \rightarrow \varepsilon$

Anatomy of a Grammar

$$S \rightarrow 0S1 \mid R$$

$$R \rightarrow \varepsilon$$

What are the different kinds of symbols here?

- ▶ **Variables** (non-terminals): S, R – appear on the left-hand side, can be replaced
- ▶ **Terminals**: $0, 1$ – the output symbols, they stay in the final string
- ▶ **Start variable**: S – where generation begins

The variables are internal working symbols. The terminals form the alphabet of the generated language.

How Does G_1 Generate Strings?

Start with the start variable. Apply rules to replace variables, one at a time.

Derivation of 0011:

$$\begin{aligned} S &\Rightarrow 0S1 \\ &\Rightarrow 00S11 \\ &\Rightarrow 00R11 \\ &\Rightarrow 00\varepsilon 11 = 0011 \end{aligned}$$

Keep substituting until only **terminals** remain. The result is a **generated string**.

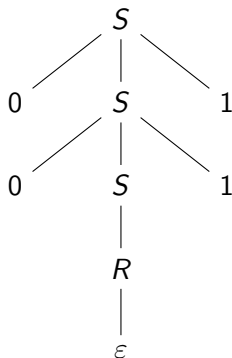
G_1 :

$S \rightarrow 0S1 \mid R$

$R \rightarrow \varepsilon$

Parse Tree for 0011

The same derivation as a **parse tree**:



G_1 :

$S \rightarrow 0S1 \mid R$

$R \rightarrow \varepsilon$

- ▶ Root = start variable
- ▶ Internal nodes = variables
- ▶ Leaves (left to right) = the generated string

What Language Does G_1 Generate?

Some strings we can derive:

- ▶ $S \Rightarrow R \Rightarrow \varepsilon$ ($n = 0$)
- ▶ $S \Rightarrow 0S1 \Rightarrow 0R1 \Rightarrow 01$ ($n = 1$)
- ▶ $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 00R11 \Rightarrow 0011$ ($n = 2$)

G_1 :

$S \rightarrow 0S1 \mid R$

$R \rightarrow \varepsilon$

Can anyone see the pattern?

$$L(G_1) = \{0^n 1^n \mid n \geq 0\}$$

This is the language we **proved is not regular!**

A simple grammar with three rules can generate it.

Formal Definition

A **context-free grammar** is a 4-tuple $G = (V, \Sigma, R, S)$.

Compare with a DFA: $(Q, \Sigma, \delta, q_0, F)$. What do the four parts of a CFG correspond to?

- ▶ V : finite set of **variables** (non-terminals)
- ▶ Σ : finite set of **terminals** (alphabet), with $V \cap \Sigma = \emptyset$
- ▶ R : finite set of **rules**, each of the form $A \rightarrow w$
where $A \in V$ and $w \in (V \cup \Sigma)^*$
- ▶ $S \in V$: the **start variable**

Derivation Notation

- ▶ $u \Rightarrow v$ (“ u **yields** v ”)
 u can become v by one substitution step
- ▶ $u \Rightarrow^* v$ (“ u **derives** v ”)
 u can become v in zero or more steps
- ▶ A sequence $u \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \cdots \Rightarrow v$ is called a **derivation** of v from u

Language of G :

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

All strings of terminals that can be derived from the start variable.

Context-Free Languages

Definition: A language A is **context-free** if there exists a CFG G such that $A = L(G)$.

Why “context-free”?

Every rule has a **single variable** on the left-hand side: $A \rightarrow w$

You can replace A **regardless of what surrounds it**.

It doesn't matter what's to the left or right of A .

In contrast, a *context-sensitive* grammar allows rules like $\alpha A \beta \rightarrow \alpha w \beta$, where the replacement depends on the surrounding symbols α, β .

Not context-free:

$B1 \rightarrow 1B$

B can only be replaced when followed by 1

Check-in: Which are Valid CFGs?

C₁:

$B \rightarrow 0B1 \mid \varepsilon$

$B1 \rightarrow 1B$

$0B \rightarrow 0B$

C₂:

$S \rightarrow 0S \mid S1$

$R \rightarrow RR$

C₁: Not a valid CFG.

$B \rightarrow 0B1 \mid \varepsilon$ is fine, but $B1 \rightarrow 1B$ and $0B \rightarrow 0B$ have more than a single variable on the left. $\leftarrow$ context-sensitive rules

C₂: Valid CFG. Every rule has a single variable on the left.

But what is $L(C_2)$?

$L(C_2) = \emptyset!$ Every rule for S keeps S in the string ($0S, S1$), so you can never eliminate all variables. Still a valid CFG!

Grammar G_2 : Arithmetic Expressions

A grammar for arithmetic expressions over $\Sigma = \{a, +, \times, (,)\}$:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T \times F \mid F$$

$$F \rightarrow (E) \mid a$$

How many rules? What are the variables, terminals, start variable?

6 rules (each “ $\mid$ ” hides a second rule).

- ▶ **Variables:** E, T, F
- ▶ **Terminals:** $a, +, \times, (,)$
- ▶ **Start variable:** E

Derivation of “ $a + a \times a$ ”

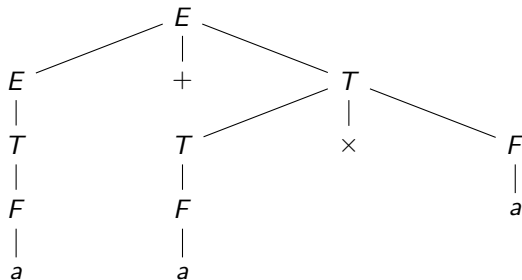
Start with E . Always substitute the **leftmost variable** first.

$$\begin{aligned} E &\Rightarrow E + T \\ &\Rightarrow T + T \\ &\Rightarrow F + T \\ &\Rightarrow a + T \\ &\Rightarrow a + T \times F \\ &\Rightarrow a + F \times F \\ &\Rightarrow a + a \times F \\ &\Rightarrow a + a \times a \end{aligned}$$

G_2 :

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T \times F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

Parse Tree for “ $a + a \times a$ ”



G_2 :

$E \rightarrow E + T \mid T$

$T \rightarrow T \times F \mid F$

$F \rightarrow (E) \mid a$

Key: $\times$ appears **deeper** than $+$ in the tree.

The grammar encodes **operator precedence**!

This is exactly what a compiler uses to determine the meaning of an expression.

Grammar G_3 : Even-Length Palindromes

Grammar over $\Sigma = \{0, 1\}$:

$$S \rightarrow 0S0 \mid 1S1 \mid \varepsilon$$

Derivation of 011110:

$$S \Rightarrow 0S0 \Rightarrow 01S10 \Rightarrow 011S110 \Rightarrow 011\varepsilon110 = 011110$$

$$L(G_3) = \{ww^R \mid w \in \{0, 1\}^*\}$$

Context-free but **not regular**. We will build a PDA for this later!

Aside: How would you extend G_3 to also generate *odd*-length palindromes?

Add $S \rightarrow 0 \mid 1$ (a single middle character, no matching needed).

Ambiguity in Natural Language

Consider the English sentence:

“The boy saw the girl with the mirror.”

How many different meanings can you find?

1. The boy used the mirror to see the girl (instrument)
2. The boy saw the girl who had the mirror (possession)

In natural language, ambiguity is common and resolved by context. In programming languages, ambiguity is **undesirable**: code should have exactly one meaning.

An Ambiguous Grammar (G_4)

Consider a simpler grammar for arithmetic:

$$G_4 : \quad E \rightarrow E + E \mid E \times E \mid (E) \mid a$$

G_4 has **one variable** instead of three. But it generates the **same language** as G_2 :

$$L(G_4) = L(G_2)$$

G_2 :

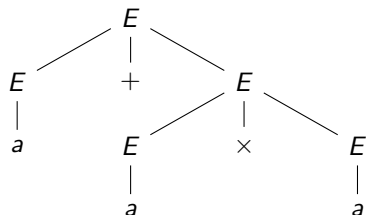
$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T \times F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

Looks simpler. So what's the problem?

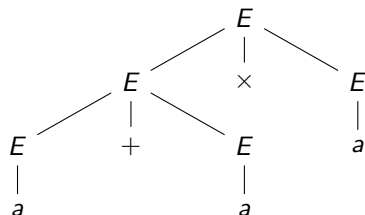
Try deriving $a + a \times a$ in G_4 . Can you find **two different** parse trees?

Two Parse Trees for " $a + a \times a$ " in G_4

Tree 1: $a + (a \times a)$



Tree 2: $(a + a) \times a$



Same string, same grammar, **two different parse trees**.
Different trees $\Rightarrow$ different meanings $\Rightarrow$ the grammar is **ambiguous**.

Ambiguity: Definition and Resolution

Definition: A grammar is **ambiguous** if some string has two or more different parse trees (equivalently, two or more leftmost derivations).

- ▶ G_4 ($E \rightarrow E + E \mid E \times E \mid (E) \mid a$) is **ambiguous**
- ▶ G_2 ($E \rightarrow E + T \mid T, \quad T \rightarrow T \times F \mid F, \quad F \rightarrow (E) \mid a$) is **unambiguous**
- ▶ Same language, different grammars!

How did G_2 avoid ambiguity?

Multiple variables (E, T, F) force a unique grouping that encodes operator precedence. This is a standard compiler design technique.

Inherently ambiguous: A CFL is inherently ambiguous if *every* grammar for it is ambiguous. These exist, but are rare.

Break

Back in 10 minutes

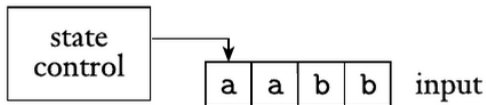
From DFA to PDA

DFAs can't recognise $\{0^n1^n\}$: they have **no memory** beyond the current state.

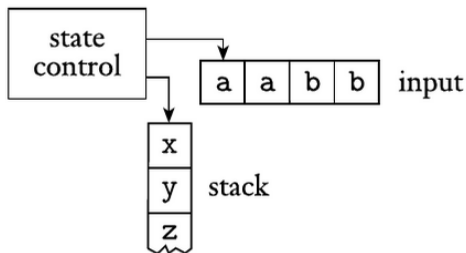
Idea: Take a finite automaton and give it access to a **stack**.

	DFA	PDA
States	finite set	finite set
Input	read left to right	read left to right
Memory	current state only	current state + stack
Determinism	deterministic	non-deterministic
Recognises	regular languages	context-free languages

PDA: Schematic View



Schematic for a **finite automaton**: state control reads input via a



head.

Pushdown automaton: same thing, plus a **stack**.

The Stack

A stack is a **last-in, first-out (LIFO)** data structure:

- ▶ **Push**: add a symbol to the top
- ▶ **Pop**: remove and read the top symbol
- ▶ Can only access the **top** (not deeper elements)
- ▶ **Unbounded**: can grow as large as needed

Practical trick: Push a special **bottom-of-stack marker** (\$) at the start. If you ever see \$ on top, the stack is effectively empty. (The formal definition doesn't include "test if empty" as a primitive, but \$ gives you the same effect.)

PDA for $\{0^n 1^n \mid n \geq 0\}$

Strategy: Read 0s and **push** each onto the stack. When you see a 1, **pop** a 0 for each 1. Accept if the stack is empty at the end.

Trace on input 0011:

Step	Read	Action	Stack (top $\rightarrow$)
1	0	push 0	0
2	0	push 0	00
3	1	pop 0, match	0
4	1	pop 0, match	(empty)
	end	stack empty $\rightarrow$ accept	

This PDA is **deterministic**: no guessing needed.

PDA for $\{ww^R \mid w \in \{0,1\}^*\}$: The Idea

Remember G_3 generated $\{ww^R\}$. Now let's build a **machine** for the same language.

Problem: The machine reads left to right. It doesn't know how long the input is, so it **can't know where the middle is**.

G_3 :

$S \rightarrow 0S0$

$S \rightarrow 1S1$

$S \rightarrow \epsilon$

Strategy (using non-determinism):

1. Read symbols, **push** each onto the stack
2. At each step, non-deterministically **guess**: am I at the midpoint?
3. If yes: switch to **popping**, matching each popped symbol against the input
4. **Accept** if the stack is empty at the end

Many branches guess wrong and fail. Only the correct guess leads to acceptance.

PDA for $\{ww^R\}$: Trace on 011110

Step	Read	Action	Stack (top $\rightarrow$)
1	0	push 0	0
2	1	push 1	1 0
3	1	push 1	1 1 0
		guess: midpoint!	1 1 0
4	1	pop 1, match ✓	1 0
5	1	pop 1, match ✓	0
6	0	pop 0, match ✓	(empty)
	end	stack empty $\rightarrow$ accept	

Other branches (guessing midpoint at step 1 or 2) would hit mismatches and reject.

Same string we derived from grammar G_3 earlier!

Non-determinism in PDAs

When a PDA forks non-deterministically, the **entire machine replicates**:

- ▶ Current state, head position, **and the full stack** are all copied
- ▶ Each branch proceeds independently
- ▶ The machine **accepts** if *any* branch reaches an accept state

Key difference from finite automata:

DFA vs NFA	Same power (DFA = NFA)
DPDA vs NPDA	Different power (DPDA $\subsetneq$ NPDA)

$\{ww^R\}$ requires non-determinism: the machine must **guess** the midpoint.

A deterministic PDA **cannot** recognise this language.

PDA: Formal Definition

A **pushdown automaton** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$.

Compare with a DFA: $(Q, \Sigma, \delta, q_0, F)$. What's new?

- ▶ Q : finite set of states
- ▶ Σ : input alphabet
- ▶ Γ : **stack alphabet** (can differ from Σ)
- ▶ $\delta : Q \times \Sigma_{\varepsilon} \times \Gamma_{\varepsilon} \rightarrow \mathcal{P}(Q \times \Gamma_{\varepsilon})$: transition function
- ▶ $q_0 \in Q$: start state
- ▶ $F \subseteq Q$: accepting states

Where $\Sigma_{\varepsilon} = \Sigma \cup \{\varepsilon\}$ and $\Gamma_{\varepsilon} = \Gamma \cup \{\varepsilon\}$.

$\mathcal{P}(\cdot)$ = power set $\Rightarrow$ **non-determinism** (multiple possible moves).

Reading the Transition Function

$$\delta(q, a, c) = \{(r_1, d), (r_2, e)\}$$

In state q , reading input a , popping c from the stack:

- ▶ Move to r_1 and push d , **or**
- ▶ Move to r_2 and push e

Shorthand for transitions: $a, b \rightarrow c$

Read a from input ($a = \varepsilon$: don't read)

Pop b from stack ($b = \varepsilon$: don't pop)

Push c onto stack ($c = \varepsilon$: don't push)

PDA Acceptance

A PDA **accepts** string w if there exists **some** sequence of transitions that:

1. Starts in q_0 with an empty stack
2. Reads all of w
3. Ends in a state in F

Just like NFAs: at least one accepting path is enough.

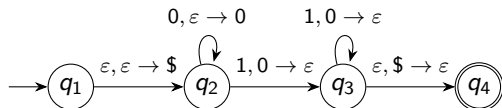
The **language** of a PDA P :

$$L(P) = \{w \in \Sigma^* \mid P \text{ accepts } w\}$$

Formal PDA for $\{0^n 1^n \mid n \geq 0\}$

$Q = \{q_1, q_2, q_3, q_4\}$, $F = \{q_4\}$

$\Sigma = \{0, 1\}$, $\Gamma = \{0, \$\}$



Reading the transitions:

- ▶ $q_1 \rightarrow q_2$: push \$ marker
- ▶ q_2 loop: push 0 for each 0 read
- ▶ $q_2 \rightarrow q_3$: first 1 pops a 0
- ▶ q_3 loop: pop 0 for each 1
- ▶ $q_3 \rightarrow q_4$: see \$, accept

Deterministic: every configuration has at most one possible move.

Transition Function for $\{0^n 1^n\}$ PDA

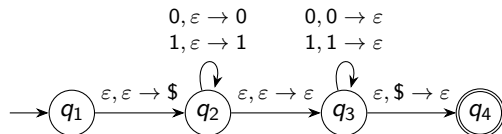
δ is given by the following table, wherein blank entries signify $\emptyset$.

Input: Stack:	0			1			ϵ		
	0	\$	ϵ	0	\$	ϵ	0	\$	ϵ
q_1									$\{(q_2, \$)\}$
q_2			$\{(q_2, 0)\}$	$\{(q_3, \epsilon)\}$					
q_3				$\{(q_3, \epsilon)\}$				$\{(q_4, \epsilon)\}$	
q_4									

Blank entries signify $\emptyset$ (no valid transition). Each non-empty entry gives the set of possible (state, push symbol) pairs.

Formal PDA for $\{ww^R \mid w \in \{0,1\}^*\}$

$$Q = \{q_1, q_2, q_3, q_4\}, \quad F = \{q_4\}$$
$$\Sigma = \{0, 1\}, \quad \Gamma = \{0, 1, \$\}$$



Reading the transitions:

- ▶ $q_1 \rightarrow q_2$: push \$ marker
- ▶ q_2 loop: push each input symbol
- ▶ $q_2 \rightarrow q_3$: **guess midpoint!**
($\epsilon, \epsilon \rightarrow \epsilon$: no read, no pop, no push)
- ▶ q_3 loop: pop and match
- ▶ $q_3 \rightarrow q_4$: see \$, accept

Non-deterministic: the $q_2 \rightarrow q_3$ transition ($\epsilon, \epsilon \rightarrow \epsilon$) can be taken at any time.

The Key Theorem: $\text{CFG} = \text{PDA}$

Theorem: A language is context-free if and only if some pushdown automaton recognises it.

Two directions:

- ▶ **CFG $\rightarrow$ PDA:** Given a CFG, build a PDA for the same language.

We will prove this direction.

- ▶ **PDA $\rightarrow$ CFG:** Given a PDA, build a CFG for the same language.

True, but the construction is more involved. (Sipser Ch. 2.2; know the fact, not the proof.)

Together: $A \text{ is a CFL} \iff \exists \text{ CFG } G \text{ with } A = L(G) \iff \exists \text{ PDA } P \text{ with } A = L(P).$

CFG $\rightarrow$ PDA Construction

Given a CFG G , build a PDA that simulates a leftmost derivation on the stack.

Key insight: Only substitute variables when they are at the **top** of the stack. When a terminal appears on top, pop it and match it against the input immediately.

Algorithm:

1. **Push** the start variable onto the stack
2. If top is a **variable**: replace it with a right-hand side (non-deterministically choose which rule to apply)
3. If top is a **terminal**: pop it and **match** against the next input symbol (reject this branch if they disagree)
4. If the stack is **empty**: accept

Trace: " $a + a \times a$ " with G_2

Stack (top first)	Action	Input read
E	push start variable	
$E + T$	substitute $E \rightarrow E + T$	
$T + T$	substitute $E \rightarrow T$	
$F + T$	substitute $T \rightarrow F$	
$a + T$	substitute $F \rightarrow a$	
$+ T$	match $a \checkmark$	a
T	match $+ \checkmark$	$a +$
$T \times F$	substitute $T \rightarrow T \times F$	$a +$
$F \times F$	substitute $T \rightarrow F$	$a +$
$a \times F$	substitute $F \rightarrow a$	$a +$
$\times F$	match $a \checkmark$	$a + a$
F	match $\times \checkmark$	$a + a \times$
a	substitute $F \rightarrow a$	$a + a \times$
(empty)	match $a \checkmark \rightarrow$ accept	$a + a \times a$

G_2 :

$E \rightarrow E + T \mid T$

$T \rightarrow T \times F \mid F$

$F \rightarrow (E) \mid a$

The non-determinism handles the choice of which rule to apply.
Only the correct choices lead to acceptance.

Completing the Picture

PDA $\rightarrow$ **CFG**: Also true, but more involved. (Sipser Ch. 2.2; know the fact, not the proof.)

Is every regular language also context-free?

Yes! A DFA is just a PDA that **never uses its stack**. Since PDAs recognise exactly the CFLs, every regular language is context-free.

$$\text{Regular} \subsetneq \text{Context-Free}$$

(Strict: $\{0^n 1^n\}$ is context-free but not regular.)

	Recogniser	Generator
Regular	DFA / NFA	Regular expressions
Context-free	PDA	CFG

Today's Journey

- ▶ Regular languages are **limited**: a DFA has no memory beyond its current state
- ▶ **Question**: What if we give a finite automaton some memory?
- ▶ Two answers, two perspectives:
 - ▶ **CFGs**: generate strings via substitution rules, derivations, parse trees
 - ▶ **PDAs**: recognise strings using a DFA + stack
- ▶ The big result: **CFGs and PDAs are equivalent**. They describe exactly the same class of languages: the **context-free languages**.
- ▶ Every regular language is context-free, but not vice versa.

The Examples That Tied It Together

- ▶ $\{0^n 1^n\}$: proved **not regular** (pumping lemma) $\rightarrow$ grammar G_1 generates it $\rightarrow$ deterministic PDA recognises it
- ▶ $\{ww^R\}$: grammar G_3 generates it $\rightarrow$ PDA recognises it, but **must guess the midpoint** (non-deterministic)
- ▶ Arithmetic expressions ($a + a \times a$):
 - ▶ G_2 (3 variables): unambiguous, encodes operator **precedence**
 - ▶ G_4 (1 variable): ambiguous, two parse trees for the same string
- ▶ CFG $\rightarrow$ PDA construction: traced $a + a \times a$ through the stack, simulating the derivation from G_2

Comparing the Two Levels

	Regular	Context-Free
Machine	DFA / NFA	PDA
Descriptive	Regex	CFG
Memory	None (finite states)	Stack (unbounded)
Det = Non-det?	Yes (DFA = NFA)	No (DPDA $\subsetneq$ NPDA)

Same **pattern** at both levels: a machine model and a descriptive model, proved equivalent. But context-free languages lose some nice closure properties.

Looking Ahead

- ▶ $\{a^n b^n c^n \mid n \geq 0\}$ is **not context-free**.
A stack can match *two* things (*a*'s against *b*'s), but not *three*.
- ▶ **Turing machines**: What if we replace the stack with **random-access memory**?
That's the next (and most general) computational model.

Textbook reading: Sipser, *Introduction to the Theory of Computation*
Ch. 2.1 (CFGs), Ch. 2.2 (PDAs & equivalence), Ch. 2.3 (non-CFLs)